

Swift Fundamentals – Loops, Control Flow, Functions, Collections, Structs & Classes

Learning Objectives

By the end of this session, learners will be able to:

- Use control flow to make decisions.
 - Iterate over data using loops.
 - Create reusable functions.
 - Work with Arrays, Dictionaries, and Sets.
 - Model data using Structures.
 - Understand Classes and the difference between value and reference types.
-

1. Control Flow

Control flow determines how the program executes based on conditions or repeated actions.

if Statement

Used when a block of code should execute only if a condition is true.

```
let age = 20

if age >= 18 {
    print("Eligible to vote")
}
```

if-else Statement

Executes one block if the condition is true and another if it is false.

```
let marks = 65

if marks >= 40 {
    print("Pass")
} else {
    print("Fail")
}
```

else-if Ladder

Useful when multiple conditions need to be evaluated.

```
let score = 82

if score >= 90 {
    print("Grade A")
} else if score >= 75 {
    print("Grade B")
} else if score >= 50 {
    print("Grade C")
} else {
    print("Fail")
}
```

switch Statement

Preferred when comparing one value against many possible cases.

```
let day = 3

switch day {
case 1:
    print("Monday")
case 2:
    print("Tuesday")
case 3:
    print("Wednesday")
default:
    print("Invalid Day")
}
```

Benefits

- Cleaner than long if-else chains.
- Exhaustive checking.
- Supports ranges and tuples.

Example:

```
let marks = 85

switch marks {
case 90...100:
    print("Excellent")
}
```

```
case 75..<90:
    print("Good")
case 40..<75:
    print("Average")
default:
    print("Fail")
}
```

guard Statement

Used for early exit and improves code readability.

```
func login(username: String?) {

    guard let username = username else {
        print("Username missing")
        return
    }

    print("Welcome \$(username)")
}
```

Use `guard` when the remaining code depends on a condition being true.

2. Loops

Loops repeat a block of code.

for-in Loop

```
for number in 1...5 {
    print(number)
}
```

Output

```
1
2
3
4
5
```

Loop through an array

```
let fruits = ["Apple", "Banana", "Orange"]

for fruit in fruits {
    print(fruit)
}
```

Using enumerated()

```
for (index, value) in fruits.enumerated() {
    print(index, value)
}
```

while Loop

Runs while a condition remains true.

```
var number = 1

while number <= 5 {
    print(number)
    number += 1
}
```

repeat-while Loop

Executes at least once before checking the condition.

```
var count = 1

repeat {
    print(count)
    count += 1
} while count <= 5
```

break

Stops loop execution.

```
for number in 1...10 {  
    if number == 5 {  
        break  
    }  
    print(number)  
}
```

continue

Skips the current iteration.

```
for number in 1...5 {  
    if number == 3 {  
        continue  
    }  
    print(number)  
}
```

Practice Questions

- Print numbers from 1 to 100.
 - Print even numbers.
 - Reverse counting.
 - Multiplication table.
 - FizzBuzz.
 - Find the largest number in an array.
-

3. Functions

Functions group reusable logic.

Simple Function

```
func greet() {  
    print("Welcome to Swift")  
}  
  
greet()
```

Function with Parameters

```
func greet(name: String) {  
    print("Hello \(name)")  
}  
  
greet(name: "Alex")
```

Function Returning Value

```
func add(_ a: Int, _ b: Int) -> Int {  
    return a + b  
}  
  
let result = add(10, 20)
```

Default Parameters

```
func greet(name: String = "Guest") {  
    print(name)  
}  
  
greet()  
greet(name: "Mahesh")
```

External Parameter Names

```
func transfer(amount: Double, to account: String) {  
    print("Transferred \(amount) to \(account)")  
}
```

```
transfer(amount: 500, to: "Savings")
```

Variadic Parameters

```
func sum(numbers: Int...) -> Int {  
    numbers.reduce(0, +)  
}  
  
sum(numbers: 10,20,30)
```

inout Parameters

```
func swapNumbers(_ a: inout Int, _ b: inout Int) {  
  
    let temp = a  
    a = b  
    b = temp  
}  
  
var x = 10  
var y = 20  
  
swapNumbers(&x, &y)
```

Practice Questions

- Calculator
- Prime Number
- Factorial
- Fibonacci
- Temperature Converter

4. Collections

Collections store multiple values.

Swift provides:

- Array

- Dictionary
 - Set
-

Arrays

Ordered collection.

```
var numbers = [10,20,30]
```

Common Operations

```
numbers.append(40)

numbers.remove(at: 1)

numbers[0] = 100

print(numbers.count)

print(numbers.contains(30))
```

Loop through array

```
for value in numbers {
    print(value)
}
```

Dictionary

Stores key-value pairs.

```
var employee = [

    "id":101,

    "name":"John"

]
```

Access value


```
print(employee["name"] ?? "")
```

Update

```
employee["city"] = "Bangalore"
```

Set

Stores unique values.

```
var cities: Set = [  
    "Delhi",  
    "Mumbai",  
    "Delhi"  
]
```

Output

```
Delhi  
Mumbai
```

Set Operations

```
let a: Set = [1,2,3]  
  
let b: Set = [3,4,5]  
  
a.union(b)  
  
a.intersection(b)  
  
a.subtracting(b)
```

Higher-Order Functions

map

```
let numbers = [1,2,3]

let doubled = numbers.map { $0 * 2 }
```

filter

```
let even = numbers.filter {
    $0 % 2 == 0
}
```

reduce

```
let total = numbers.reduce(0, +)
```

sorted

```
let sorted = numbers.sorted(by: >)
```

Practice Questions

- Remove duplicates.
- Find highest salary.
- Student marks system.
- Employee directory.

5. Structures

A Structure is a value type used to model data.

```
struct Employee {

    var id: Int
```

```
    var name: String

    var salary: Double
}
```

Create object

```
let emp = Employee(
    id: 1,
    name: "John",
    salary: 50000
)
```

Methods

```
struct Rectangle {

    var width: Double

    var height: Double

    func area() -> Double {

        width * height
    }
}
```

Computed Property

```
struct Circle {

    var radius: Double

    var area: Double {

        3.14 * radius * radius
    }
}
```

Mutating Function

```
struct Counter {  
  
    var value = 0  
  
    mutating func increment() {  
  
        value += 1  
    }  
}
```

6. Classes

Classes are reference types.

```
class Employee {  
  
    var name: String  
  
    init(name: String) {  
        self.name = name  
    }  
}
```

Object Creation

```
let emp = Employee(name: "Alex")
```

Methods

```
class BankAccount {  
  
    var balance = 0  
  
    func deposit(amount: Int) {  
  
        balance += amount  
    }  
}
```

```
}  
}
```

Reference Type Example

```
class Person {  
    var name = ""  
}  
  
let p1 = Person()  
  
p1.name = "Alice"  
  
let p2 = p1  
  
p2.name = "Bob"  
  
print(p1.name)
```

Output

Bob

Both variables point to the same object.

Struct vs Class

Feature	Struct	Class
Type	Value Type	Reference Type
Memory	Copied on assignment	Shared reference
Inheritance	Not supported	Supported
Mutability	Requires mutating	Methods modify properties directly
Recommended Use	Models, immutable data	Shared state, complex objects

Mini Project

Student Management System

Create a console application that can:

- Add a student
- Display all students
- Search by ID
- Update marks
- Remove a student
- Calculate average marks
- Display the topper

Concepts to use:

- Functions
 - Arrays
 - Structures
 - Control Flow
 - Loops
 - Dictionaries (optional)
-

Summary

After completing this session, learners should be able to:

- Make decisions using `if`, `guard`, and `switch`.
- Repeat tasks using `for`, `while`, and `repeat-while`.
- Write reusable functions with parameters and return values.
- Store and manipulate data using Arrays, Dictionaries, and Sets.
- Use higher-order functions like `map`, `filter`, and `reduce`.
- Create data models using `struct`.
- Understand when to use `class` versus `struct`.